The functions in this sub-section are similar to functions we've already seen. `strpos()`, `str_contains()`, etc., can be used to test if sub-strings exist, but these functions differ in that they will return portions of the input string rather than positional values. These extraction functions use more memory than their search counterparts, so only use them if you care about the extracted data.

substr(), mb_substr(), mb_str_cut()

Let's start with `substr()`, which is a fine-grained tool for extracting a sub-string based on byte lengths. The function takes an input string as the first argument, an integer offset value as the second, and an optional integer length value as the third. If offset is negative, then bytes are counted from the end of the input string, and if it exceeds the length of the string, then an empty string is returned. If length is negative, then that many bytes are omitted from the end of the returned string, and if that means the offset is exceeded, then you get an empty string again. It is possible to specify a length of zero (resulting in an empty string return) or even null (which has the same effect as passing no length argument at all: the remainder of the string is returned).

```
$input = "An\x000utside Context Problem was the sort of thing most
civilisations encountered just once";

print substr($input, 3, 23) . PHP_EOL;

Output:
Outside Context Problem
```

And there we have it, fairly straightforward stuff. We told `substr()` to extract twenty-three bytes, starting with the byte at position three (remember we're counting from zero; think of it like a byte array). The fourth byte is "O", just after the null byte, and we start counting length from there. The null byte is included here to demonstrate binary safety. Do not use this function on multi-byte data, because you risk chopping through the middle of a valid multi-byte sequence:

```
$name = "Hello, \u{141}ukasz!"; // \u{141} = UTF-8 0xC5 0x81

print substr($name, -7) . PHP_EOL;

Output:
?ukasz!
```